AgentForge Studio: Self Optimized LLM Multi Agent System for Software Engineering Workflows

R. M. K. N. Gunasena
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22078582@my.sliit.lk

P. A. R. B. Subasinha
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22098450@my.sliit.lk

R. H. F. Hamna
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22516916@my.sliit.lk

R. M. M. P. Bandara
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22249166@my.sliit.lk

Prof. Nuwan Kodagoda
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
nuwan.k@sliit.lk

Mr. Eishan Weerasinghe
Faculty of Computing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
Eishan.w@sliit.lk

Abstract - Large language models can now write most of a web application, yet the agent's own report that the work is finished remains the weakest evidence in the process. This paper presents AgentForge Studio, a desktop system in which four LLM agent roles (requirements and design, build, quality assurance and deployment) carry a product idea through an adaptive interview, an approved plan, an IEEE-formatted requirements specification with eleven kinds of diagrams, wireframes, an interactive prototype, a phased build, browser-level testing and a cloud deployment, with a human in the loop at every stage. The roles share one tool-calling engine and hand work to each other through a project record on disk. Every hand-off is checked by code rather than by a model: a depth floor on the plan; schema, plan-coverage and evidence audits on the specification; a named-page gate on the build; requirement verdicts computed from test-runner output; and a probe of the deployed HTTPS address. Personal data is replaced with placeholders before every model request. Finished applications can be edited by pointing at elements, extended with new features, and deployed only after explicit human approval. Measurements taken during development show where a long-running agent pipeline breaks and what fixed each break.

Index Terms- autonomous software engineering, large language models, multi-agent systems, requirements engineering, software testing, deployment automation, data privacy

I. Introduction

Ask one model to build a thirty-page web application in a single conversation and it tends to return a smaller one. In AgentForge Studio's early runs the model said as much: partway through, it decided it "must be honest about capacity" and finished a coherent subset of the approved pages. No file it wrote was wrong; the application was simply smaller than the one the user had approved, and only a count against the plan could show by how much.

Code-trained language models have brought function- and module-level programming within reach of automation [1], [2], and agentic systems now resolve a measurable share of real repository issues end to end [3], [4]. The hard part has moved: it is now the distance between code that runs and an application someone can trust. Surveys of LLM-based multi-agent systems for software engineering name quality assurance and evaluation as the open problems [5].

AgentForge Studio is built on one rule: a model may write anything, but code decides whether a stage is finished. Four agent roles take turns on one tool-calling engine and pass work to each other through files in the project's own record; every stage boundary has a check that reads those files and either lets the project through or names what is fully or partially missing. The model is used where judgement is needed and kept out of the decision about whether that work is done. The paper makes four contributions:

1) A pipeline from a one-sentence idea, given as text, attachments or voice, to a live HTTPS address, with four agent roles sharing one engine and one on-disk record (Section III).
2) Deterministic checks at each hand-off, including requirement status computed from test-runner output rather than from the agent's report (Section III-E).
3) A privacy layer that keeps typed personal data out of model requests while the artifacts keep the real values (Section III-G).
4) Measurements of where a long-running agent pipeline broke and what fixed each break, with a protocol for a controlled evaluation (Section V).

II. Literature review

A. Code Generation and Repository Agents

HumanEval-style benchmarks test whether a model can turn a docstring into a working function [1], and SWE-bench whether an agent can resolve a real GitHub issue [3]; open-weight families such as Qwen2.5-Coder [2] and Gemma 3 [6] run on one workstation. ReAct set the pattern of interleaved reasoning and tool calls [7], SWE-agent showed that the agent's interface changes how often it succeeds [4], and Agentless reached competitive results with a fixed localise, repair and validate pipeline [8]. AgentForge Studio goes further: the stage order and every stop condition are fixed in code.

B. Multi-Agent Software Engineering

Dividing a problem among communicating specialists predates language models [9], [10]. ChatDev runs a virtual software company through chat-chained phases [11], MetaGPT encodes standard operating procedures as roles exchanging documents [12], AutoGen frames collaboration as multi-agent conversation [13], and AgentCoder separates programming

from test design and execution [14]. All four start from a requirement given as text and end with source code; interviewing the user, holding the result to an approved plan and deploying it are outside their scope (Table I).

TABLE I. CAPABILITIES AS DESCRIBED IN EACH SYSTEM'S PUBLICATION

Capability	ChatDev [11]	MetaGPT [12]	AutoGen [13]	AgentCoder [14]	AgentForge
Role-specialised LLM agents	Yes	Yes	Yes	Yes	Yes
Interviews the user for requirements	No	No	No	No	Yes
Specification checked by code against an approved plan	No	Partial	No	No	Yes
Progress decided by executed tests	Partial	Partial	Partial	Yes	Yes
Browser journeys and screenshot review	No	No	No	No	Yes
Deployment verified at a public address	No	No	No	No	Yes
Personal data hidden from the model	No	No	No	No	Yes

Partial: MetaGPT's structured documents are not checked against a plan the user approved; ChatDev, MetaGPT and AutoGen feed run errors back without a pass criterion fixed in advance.

C. Self-Checking and Long Contexts

Self-Refine loops a model over its own critique [15] and Reflexion turns failure feedback into verbal memory [16]; both let the model judge its own work. Yet models do not reliably correct their reasoning without external feedback [17], LLM judges show position, verbosity and self-enhancement biases [18], and models use the middle of a long context poorly [19]. AgentForge Studio uses model critique only where a picture or a document must be read; pass or fail is always a function of files on disk.

D. Requirements Engineering

ISO/IEC/IEEE 29148 lists the properties of a well-formed requirement, among them necessary, unambiguous, singular and verifiable [20]. Corpora such as PURE [21] and the expanded PROMISE set [22] make requirement quality measurable, and multi-agent LLM pipelines have derived UML models from natural-language requirements [23]. AgentForge Studio checks a subset of these properties mechanically.

III. METHODOLOGY

AgentForge Studio runs on the user's computer. A Studio interface talks to a Python backend, the backend keeps one engine per project, and the engine runs whichever agent role the current stage needs. Fig. 1 shows the parts and the checks between stages.

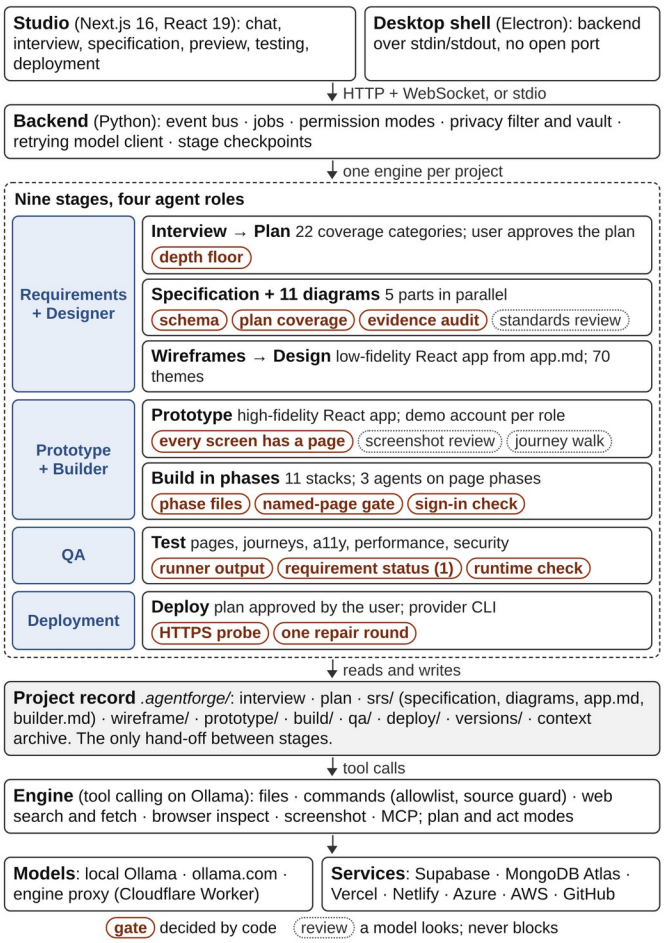


Fig. 1. AgentForge Studio architecture. Four agent roles share one engine and hand off through the project record; the checks between stages are decided by code.

A. Architecture

The pipeline has nine stages: interview, plan, specification, wireframes, design, prototype, build, test and deploy. Four agent roles own them (Table II). A role is not a separate process or model but a set of prompts, a stage of the project and a label on the events the Studio shows. Every instruction a model receives is a Markdown file under *prompts/*, so changing how a stage behaves means editing a file, not the code.

Each project keeps a record folder, *.agentforge/*, beside the application it builds: the interview, the plan, the specification, the wireframe and prototype apps, the build's phase state, the test runs and the deployment record. A later stage reads what an earlier one wrote, and a stopped stage resumes from its saved state.

TABLE II. AGENT ROLES, WHAT THEY PRODUCE AND WHAT CHECKS THEM

Role	Produces	Checked by code
Requirements + Designer	Interview, plan, specification, 11 diagrams, hand-off files, wireframes, theme	Depth floor; schema; plan coverage; evidence audit
Prototype + Builder	Prototype, demo accounts, application, unit and journey tests, build report	Every screen has a page; phase files; named-page gate; sign-in page
QA	Page and journey fixes, a11y, performance and security results	Runner output; Eq. (1); runtime check
Deployment	Plan the user approves, release, run record	HTTPS probe by the Studio; one repair

B. Engine and Context

The engine is a terminal coding agent built on Ollama's tool-calling API [24], with ten tools: list, read and search files; write a file or replace one exact span; run a command; search and fetch the web; inspect a local page in a headless browser; and take a screenshot that a vision-capable model is shown. More tools attach through the Model Context Protocol [25], and any Ollama model with tool calling can drive it.

A stage runs in plan mode, where the write and command tools are withheld and the model produces a numbered plan, then in act mode, where it carries the plan out and must end with an explicit marker: complete, blocked, verified or incomplete. The loop is bounded at eight execution rounds, three audit rounds and three rounds without a tool call, and a JSON answer the validator rejects goes back with its error, up to three times.

The first design kept one conversation per project and compacted older turns into a summary once it passed 88% of the model's window. Section V-C shows why that was not enough: each stage, and each build phase, now starts from a small conversation, earlier turns are archived verbatim, and the stage reads what it needs from the record.

C. Requirements: Interview, Plan, Specification

The interview asks one question at a time. The first, the product type, comes from a catalogue of ten and costs no model call; every later question is chosen by the model from what is still unknown across 22 requirement categories (seven critical: purpose, users, roles, core features, workflows, data and authentication), each tracked as known, partial, unknown or not applicable. A one-screen tool needs a handful of questions and a multi-role system up to 25, after which the user confirms a summary; a user who skips the interview gets a plan that states its assumptions.

The plan is what the user approves, and it is a ceiling: the specification's pages come from its screens, its tables from its records and its roles from its users. A depth floor rejects a plan that cannot be built from: the product intent must be at least 120 characters, every screen needs a purpose, every record keeps at least two fields, and at least one workflow has two or more steps. With sign-in, at least one role needs two things it can do, the plan must say how accounts are created, and open sign-up may not create a privileged role.

The standard software requirements specification is written in five parts at once (frame, tables, API, requirements and quality) from page routes and table names computed once from the plan, then joined and validated against a Pydantic schema that requires at least three functional and three non-functional requirements. The *coverage gate* then copies in what the plan states and the draft omitted, and counts what is still missing: a planned screen with no page, a record with no table, a feature, workflow or role ability with no requirement, and a functional requirement with no traceability row. Any remaining gap stops the stage.

Two quality checks follow. The *evidence audit* is seven fixed rules: both requirement sets exist; identifiers follow FR-### or NFR-### and are unique; each requirement says "shall"; each names a verification method; each functional requirement has a rationale; each non-functional requirement carries a number with a unit or a named standard; and each functional requirement has exactly one traceability row with a test case. A model reviewer then scores five qualities from 0 to 5, but Python decides that the draft passed: no blocker, each failed audit rule counting as one, and every score at least 3. Repairs are JSON edits applied only if the result still validates.

Eleven diagram kinds are then drawn in Mermaid, each in its own call and six at a time: system context, use case, entity-relationship (crow's foot), sequence, activity, class, state machine, data flow (Yourdon/DeMarco), business process (BPMN 2.0.2), component and deployment, the UML ones to UML 2.5.1. Each is rendered to SVG, photographed and reviewed by a vision model for clipped labels, crossing lines and unreadable scale, for at most two rounds. Two hand-off files, written by code so they cannot be truncated, close the stage: *app.md* holds what the user asked for and the site map; *builder.md* holds the full specification for the build, the tests and the deployment.

D. Wireframes, Design and Prototype

The wireframes and the prototype are both small React applications (Vite, TypeScript, Tailwind CSS and shadcn/ui) that the agent writes with its own file tools, following Anthropic's web-artifacts-builder skill [26]; the Studio creates each app and bundles it into one self-contained HTML page. Wireframes are drawn from *app.md* alone, so they show what the user asked for rather than what the specification inferred. For ten or more screens, one agent writes the shared layout and the first three pages, and three agents write the rest in parallel.

The user then picks one of 70 design themes, each a DESIGN.md file the prototype must follow. The prototype is a new high-fidelity app written from the approved wireframes, the theme and *app.md*, signing in with one fictitious account per role that the build later seeds. Two reviews follow: every page is photographed at 1440 and 390 pixels and a vision model lists visible defects, up to 40 of which are fixed; and each user journey (up to 24, of up to ten steps) is walked in a real browser by clicking the links the pages actually have. These are reviews, not gates.

E. Build and Verification

The builder installs one of eleven stack templates (Next.js, Vite or Remix on Supabase or MongoDB, and a MERN microservice layout) into an empty workspace, never over existing work. A preflight step checks Node.js, a browser and the stack's database before anything is planned and lists everything missing in one prompt. The model may then ask up to five questions it cannot settle itself; after that the build never stops to ask. A decision with no safe default takes the most conservative reading of the specification, and each such choice is recorded in the build report as a gap labelled *gap*, *unavailable*, *untested* or *known*.

The Studio, not the model, holds the list of phases (Fig. 2): a foundation (data models, authentication, shared layout), page phases of four approved pages each, a wiring phase with the seed data, and the unit and journey tests. Once the foundation stands, three agents build page phases in parallel, each phase in a fresh conversation. A phase is finished when its progress record names files that exist and are not empty, and is otherwise asked for again, at most twice; an approved page still not built at the end fails the build by name, as does a sign-in page that displays demo credentials.

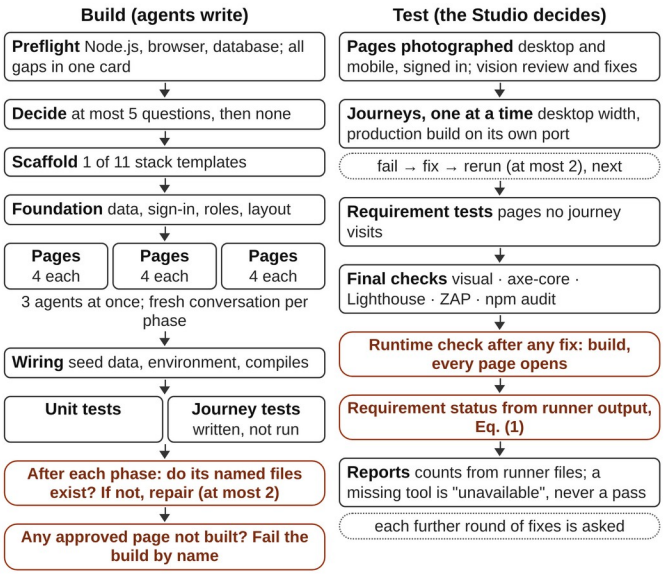


Fig. 2. Build and test sequence. Agents write the application and fix what fails; the Studio decides when a phase, the build and each requirement are done.

Testing follows a fixed order (Table III). Every page is photographed at desktop and mobile width, signed in as a role that may open it, and reviewed for visible defects. The Studio then runs each user journey on its own against the production build, with a fresh test database on MongoDB stacks; a failing journey is fixed and rerun, at most twice, before the next one, and pages no journey visits get requirement tests of their own. The agent then runs the remaining layers once each, with axe-core [27], Lighthouse [28] and ZAP [29], and a late fix is checked by building the app and opening every page. Only the Studio runs the whole journey suite, and a missing tool is recorded as unavailable, never as a pass.

Requirement status is computed, not reported. Let *P(r)* be the pages that the traceability row of requirement *r* names, *J(p)* the journeys with a step on page *p*, and *pass(j)* true when the desktop test of journey *j* passed. When no test names *r* in its title,

$$\text{met}(r) \;\Leftrightarrow\; P(r) \neq \varnothing \;\wedge\; \forall p \in P(r)\;\; \exists j \in J(p) : \text{pass}(j) \qquad (1)$$

A requirement is *failed* when some page in *P(r)* is visited only by failing journeys and *untested* otherwise. When tests do name *r*, they decide on their own. The result overwrites whatever the builder wrote and is marked as decided by the Studio.

TABLE III. TEST LAYERS AND WHAT COUNTS AS A PASS

Layer	Tool	Pass rule
Pages	Page spec, vision model	Photographed signed in; defects fixed by round
Unit	Vitest, Supertest	No failure in the runner's JSON
Journeys	Playwright	Each spec passes; phone-width failures warn
Visual	Playwright	Matches the recorded baseline
Accessibility	axe-core	No serious or critical WCAG A/AA violation
Performance	Lighthouse CI	Accessibility ≥ 0.9; others warn below 0.7–0.8
Security	ZAP baseline	No FAIL-level rule; fallback is only "partial"
Runtime	Build, smoke spec	The app builds and every page opens

F. Deployment

Deployment is planned first, like a change request (Section III-H). The targets are Vercel, Netlify, Azure App Service, AWS EC2, AWS ECS and GitHub, as the stack allows; the agent uses each provider's own command-line tool, signed in through its browser or device-code login, so no token is pasted. It reruns no tests, builds once, asks at most three questions and proves the result with at most six read-only requests. The Studio then checks the result itself: it requests the public address and every recorded check, rejects a non-HTTPS, local or private address, and retries 429 and gateway errors from a host still starting after 5 s and 10 s. A failure goes back to the agent once; if it still fails, the run ends as failed, with the cause.

G. Safety, Privacy and Reliability

Each project runs in one of four permission modes: automatic (only risky commands are asked about), manual, accept edits, and bypass, which must be switched on first. Commands are checked against an allowlist of 132 program names and a deny list (*sudo*, a download piped into a shell, Invoke-Expression, deleting the root or home directory), defense in depth rather than a sandbox, and AgentForge Studio's own source is restored if a command changes it.

Personal data does not reach a model in clear text. Everything the user types passes through Microsoft Presidio [30] with a spaCy English model [31], set to 15 entity types including Sri Lankan identity and phone numbers, and each value found is kept, encrypted, in a per-project vault under a numbered placeholder. The one client every model call passes through hides known values on the way out and restores them in the answer and in local tool arguments, while web search, web fetch and plugin tools keep the placeholders. If Presidio is missing, nothing is sent, and text shaped like a key is refused in the chat.

A transient model-service failure is retried up to ten times with back-off from 2 s to 30 s, and every stage writes a checkpoint when it starts, finishes or fails, so a run cut off by a restart resumes.

H. Human in the Loop

The agents do the work, but the user decides what is built and when the stage moves on. Nothing passes the plan, the specification, the design or a deployment without the user's approval, and every stage has a chat box and an artifact the user can change (Table IV). Questions are rationed to what only the user can decide: at most 25 in the interview, five before a build, three per change request and three per deployment. Everything else is a stated default that the user can correct.

TABLE IV. WHAT THE USER CAN DO AT EACH STAGE

Stage	User actions
Interview	Pick or type an answer; attach files; correct the summary; skip
Plan	Approve, or revise in own words; each revision is a new version
Specification	Approve; ask for a revision; answer each open decision
Wireframes	Chat; select, draw on or retype any part of a page
Design	1 of 70 themes, or 32 palettes, 117 fonts, own pictures, a brief
Prototype	Chat; select, draw on or retype; approve before the build
Build	Connect accounts; answer up to five questions; edit code
Test	Decide each further round of fixes; ask for journeys to be fixed
Deploy	Approve, revise or cancel the plan; answer up to three questions
After build	Chat changes, planned first; point at the running app

A message typed at any stage is answered from the project or applied to it, and the agent updates every artifact the request reaches without widening it. Once a specification exists, a change is planned first: the planner may ask up to three questions and shows a plan to approve, revise or cancel, and

nothing is touched before approval. Each approved change becomes a version (v1, v2, ...) with its files and the test results before and after.

Users can also point. On the wireframes, the prototype and the running application, a select tool picks an element and a pencil marks an area; the Studio describes what was picked (tag, words, attributes and markup) and attaches a screenshot for the agent. A text tool changes words in place, with no model call when they occur once in the source, and code can be edited directly in the Studio.

During a run, a permission card offers allow, allow without asking again, or deny with a note; Stop ends a model request between tokens and a command with its process tree; further rounds of fixes are a Yes or No card; and the model and its reasoning effort can be changed without losing the record.

IV. Implementation

AgentForge Studio 1.0.3 has about 32,300 lines of Python, 27,800 lines of JavaScript and 19,500 lines of Markdown prompts (Table V). The backend runs on the Python standard library with Pydantic, cryptography, Presidio and spaCy; the Studio is a Next.js 16 and React 19 application, wrapped for Windows in Electron and installed by a 150 KB online installer that checks every file against a SHA-256 manifest.

Generated applications come from 236 template files: eleven stacks, a testing kit (Playwright [32], Vitest, axe-core, Lighthouse CI and ZAP) and 18 guides. The bakery project of Section V-B used gpt-oss:120b through Ollama's cloud, and 1,733 Python unit tests and twelve Node test files cover the system with stand-in models.

TABLE V. Technology Stack

Layer	Technologies	Used for
Studio	Next.js 16, React 19, Tailwind CSS 4, Zustand 5	Chat, stage views, previews
Desktop	Electron 44, C# online installer	One window; backend over stdio
Backend	Python standard library, Pydantic 2, httpx, cryptography	API, jobs, event bus, vault
Engine	Ollama SDK (local or ollama.com), ten tools, MCP	Tool loop of every role
Privacy	Microsoft Presidio, spaCy	15 entity types hidden
Prototype kit	Vite, React 18, TypeScript, Tailwind, shadcn/ui	One bundle.html per app
Generated apps	Next.js 15, Remix 2, Vite 6, Express 4; Supabase or Mongoose 8	Eleven stacks
Generated tests	Vitest, Playwright 1.62, axe-core, Lighthouse CI, ZAP	All test layers
Cloud	Supabase, Atlas, gh, Vercel, Netlify, Azure and AWS CLIs	Provisioning, deployment

A. Runtime and Model Access

The API listens on 127.0.0.1:7824 and a WebSocket feed on 7825; in the desktop app nothing listens, and the window and the backend exchange JSON lines. Model work runs as background jobs, every event is appended to *events.jsonl* so a reconnecting browser replays the conversation, and the conversation is saved after every turn. Hosted replies are streamed, since a single response is cut off after about 100 s without a byte, and the engine proxy, a Cloudflare Worker, holds the service key, rate-limits callers and counts each call against the account's limit.

V. Evaluation and Preliminary Results

A. Research Questions and Protocol

The observations in Section V-C were gathered while the system was being built, so they cannot attribute an improvement to one component. The controlled study is the next step: a fixed suite of application ideas, from a one-screen tool to stores of over thirty screens, built with the same model and settings under each condition of Table VI, three runs per condition, with requests logged at the single model client.

TABLE VI. Research Questions, Conditions and Metrics

Research question	Condition	Primary metrics
RQ1: Do the gates make the specification more complete?	Gates on vs. first draft	Uncovered plan items; audit score
RQ2: Does a phased build deliver every page?	Phased vs. one conversation	Pages built; time; tokens
RQ3: Do agent-reported statuses match Eq. (1)?	Builder vs. Eq. (1) vs. manual test	Cohen's κ; false "met"
RQ4: What do per-phase conversations save?	Fresh vs. one compacted	Tokens; latency; total time
RQ5: How reliably is a live address reached?	Each app, each allowed target	Success rate; time; repairs
RQ6: Does the filter keep personal data out?	Filter on vs. off	Leaked values; wrongly hidden terms

B. Development Corpus

The observations come from the authors' own projects: a bakery ordering application, a 28-page application taken from specification to deployment, a 31-page online store and a store whose plan had 71 screens. Each row of Table VII is one observed run before and, where stated, after a change; the rows explain the current design and are not a comparison with other systems.

C. Observations

Table VII lists twelve failures in the order of the pipeline. On the bakery project the coverage gate took the first specification draft from one diagram to seven and from three functional requirements to six; before the changes in rows 4 to 8, the 28-page application took about seven hours of machine time from specification to deployment.

TABLE VII. Failures Observed During Development and the Change Each Led To

Failure observed	Measurement	Change made
1. One conversation builds the whole app	A coherent subset; model cited "capacity"	Numbered phases; named-page gate
2. Interview turn rewrote all facts (~4,000 tokens)	~20 s per question (up to 64 s); after: 1.9–4.6 s	Question alone; facts updated in background
3. Specification as one ~60,000-token reply	546 s, 31-page store; after: 141 s	Five parts in parallel
4. One conversation through all build phases	900,000 tokens; 21 s per call (5 s at 50,000)	Fresh conversation per phase
5. One agent draws all wireframes	About 1.5 h for 117 pages	Three agents after the shared layout
6. One agent builds all pages	2 h 37 min for 31 pages	Three agents on page phases
7. All journeys run, then fixed together	About 1.5 h, 31-page store	One journey at a time
8. Search walked test frames and packages	16 min for one search; deploy plan 37 min	Skip outputs and binaries; 20 s cap
9. Requirements marked before tests ran	0 of 116 met after all tests passed	Status from runner output, Eq. (1)
10. Routes compared with letter case	117 pages for 71 screens	Case-insensitive route keys
11. "Admin", "Require" taken for names	117-page wireframe ended with 2 pages	Names need two name-shaped words

Failure observed	Measurement	Change made
12. Screenshot added to a long conversation	2.5 MB answered; 3.4 MB got HTTP 500	Pictures only below 2.4 MB

D. Discussion

Shared files beat a shared conversation. The first design kept the project's memory in the conversation and failed on cost: by the build, every call resent hundreds of thousands of tokens. Small conversations were safe because every stage's input was already a file, in line with evidence that models use long inputs unevenly [19].

Self-reports are the weakest evidence. Rows 1 and 9 of Table VII are one failure in two stages: the model's account differed from the files. Each fix replaced a model's statement with a function of what is on disk, as models do not reliably correct themselves without external feedback [17].

A check that fires wrongly does damage. With "require" taken for a name, the agent wrote <PERSON_3>("fs") back into the code, and one capital letter in a route produced 46 duplicate pages; such checks need precision more than recall.

VI. Limitations and Future Work

Five limitations bound these claims. The evidence is internal and uncontrolled: single runs on the authors' projects with no baseline. The coverage gate matches plan items to requirements by shared words, and the audit checks a requirement's form, not its truth. Eq. (1) is only as strong as the journeys, since a journey that visits a page but asserts little still marks its requirements met. The screenshot reviews need a vision-capable model, the screenshots carry real personal data, and name detection is English-only. The system targets JavaScript web applications on Supabase or MongoDB, ships for Windows, and its command checks are not isolation. Future work is the controlled study, journey assertions tied to acceptance criteria, and agent commands run in a container.

VII. Conclusion

AgentForge Studio rests on the view that the hard part of autonomous software engineering is knowing when the work is done. It hands that decision to code at every boundary and leaves what to build, and when to move on, to the user. Building it showed that a conversation is a poor place to keep a project, and that wherever a model's report stood in for evidence, a short function over the files eventually gave the better answer. The study of Section V-A will test whether this holds beyond the projects the system was built on.

References

[1] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[2] B. Hui et al., "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[5] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, 2025, doi: 10.1145/3712003.

[6] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[8] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024.

[9] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[10] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Chichester, U.K.: Wiley, 2009.

[11] C. Qian et al., "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.

[12] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[13] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

[14] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," arXiv:2312.13010, 2023.

[15] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[17] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[18] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2023.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

[20] *Systems and Software Engineering—Life Cycle Processes—Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[21] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "PURE: A dataset of public requirements documents," in *Proc. IEEE 25th Int. Requirements Eng. Conf. (RE)*, 2017, pp. 502–505.

[22] M. Lima, V. Valle, E. Costa, F. Lira, and B. Gadelha, "Software engineering repositories: Expanding the PROMISE database," in *Proc. 33rd Brazilian Symp. Softw. Eng. (SBES)*, 2019.

[23] P. Giannouris, "Multi-agent LLM system for UML class diagram generation from natural language requirements," arXiv:2511.22409, 2025.

[24] Ollama. [Online]. Available: https://ollama.com

[25] Anthropic, "Model Context Protocol specification." [Online]. Available: https://modelcontextprotocol.io

[26] Anthropic, "web-artifacts-builder," Agent Skills repository. [Online]. Available: https://github.com/anthropics/skills

[27] Deque Systems, "axe-core: Accessibility engine for automated web UI testing." [Online]. Available: https://github.com/dequelabs/axe-core

[28] Google, "Lighthouse." [Online]. Available: https://developer.chrome.com/docs/lighthouse

[29] ZAP by Checkmarx (formerly OWASP ZAP), "Zed Attack Proxy." [Online]. Available: https://www.zaproxy.org

[30] Microsoft, "Presidio: Data protection and de-identification SDK." [Online]. Available: https://github.com/microsoft/presidio

[31] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength natural language processing in Python," 2020, doi: 10.5281/zenodo.1212303.

[32] Microsoft, "Playwright." [Online]. Available: https://playwright.dev